# T3T4 layer visualization RNN

November 29, 2020

T3T4 layer visualization

This notebook is provided to show the way to see the built model from autoBioSeqpy. A part of the functions in this notebook is provided in the script "tool/layerPlot.py" as well.

NOTE: The script used 3rd module UMAP for layer plotting.

1. Preparation

In this notebook, few packages are necessary.

Assuming that we are in 'root-of-autoBioSeqpy/notebook/T3T4', then we need to add the search path and import the modules.

Please install UMAP, matplotlib, tensorFlow and keras before using this notebook.

[1]:
```
import os, sys
sys.path.append('../../tool/libs')
sys.path.append('../..')
sys.path.append('..')
oriPath = os.path.curdir
import paraParser
import moduleRead
import dataProcess
#import analysisPlot
import numpy as np
#from sklearn.metrics import␣
↪accuracy_score,f1_score,roc_auc_score,recall_score,precision_score,confusion_matrix,matthew
↪
import tensorflow as tf
from utils import TextDecorate, evalStrList
from keras.models import Model
from keras.models import Sequential

import umap
import umap.plot
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages
```

```
Using TensorFlow backend.
```

Using UMAP is a simple way for model visualization, but still we need the model as well as the data for generating the predicted data for plotting. Thus, here we need few steps to get a built model and the related data.

1.1 Model building

As the first step, we need a model from running.py, users could build a model by using the command line below in the root of autoBioSeqpy:

python running.py –dataType protein –dataEncodingType dict –dataTrainFilePaths examples/T3T4/dataset/trainT3.txt examples/T3T4/dataset/trainT4.txt –dataTrainLabel 1 0 –dataTestFilePaths examples/T3T4/dataset/testT3.txt examples/T3T4/dataset/testT4.txt –dataTestLabel 1 0 –modelLoadFile examples/T3T4/model/RNN.py –verbose 1 –outSaveFolderPath tmpOut –savePrediction 1 –saveFig 1 –batch_size 25 –epochs 40 –shuffleDataTrain 1 –showFig 1 –modelSaveName tmpMod.json –weightSaveName tmpWeight.bin –noGPU 0 –paraSaveName parameters.txt –spcLen 1500

The outputs will be saved at your-work-path/tmpOut, including the parameters, the path of the data and constructure of the model.

Alternativly, ususally a built model is already provided in your-work-path/notebook/T3T4/models/RNN, thus users could skip this step by using the provided one.

If users would like to use their own model, please don't forget to save the model and the weight by using parameters "–modelSaveName" and "–weightSaveName".

1.2 Parameter pasering and data loading

Here we use the parameters which saved in your-work-path/notebook/T3T4/models/RNN directly. We can get the information easily by using the paraParser module.

[2]:
```
paraFile = './models/RNN/parameters.txt'
paraDict = paraParser.parseParametersFromFile(paraFile)
#print
for k in paraDict:
    print('%r: %r' %(k,paraDict[k]))
```

```
'dataType': ['protein']
'dataEncodingType': ['dict']
'spcLen': [1500]
'firstKernelSize': []
'dataTrainFilePaths': ['examples/T3T4/dataset/trainT3.txt',
'examples/T3T4/dataset/trainT4.txt']
'dataTrainLabel': [1, 0]
'dataTestFilePaths': ['examples/T3T4/dataset/testT3.txt',
'examples/T3T4/dataset/testT4.txt']
'dataTestLabel': [1, 0]
'dataTrainModelInd': []
'dataTestModelInd': []
'outSaveFolderPath': 'tmpOut'
'showFig': True
```

```
'saveFig': True
'figDPI': 300
'savePrediction': True
'dataSplitScale': None
'modelLoadFile': ['examples/T3T4/model/RNN.py']
'weightLoadFile': []
'shuffleDataTrain': True
'shuffleDataTest': False
'batch_size': 25
'epochs': 40
'useKMer': []
'KMerNum': []
'inputLength': []
'loss': 'binary_crossentropy'
'optimizer': 'optimizers.Adam()'
'metrics': ['acc', 'acc']
'modelSaveName': 'tmpMod.json'
'weightSaveName': 'tmpWeight.bin'
'noGPU': False
'paraFile': None
'paraSaveName': 'parameters.txt'
'seed': 1
'labelToMat': False
'colorText': 'Auto'
'verbose': True
'reshapeSize': []
```

Few preparing...

[3]:
```
paraDict['dataTestFilePaths'] = paraDict['dataTrainFilePaths']
paraDict['dataTestModelInd'] = paraDict['dataTrainModelInd']
paraDict['dataTestLabel'] = paraDict['dataTrainLabel']
paraDict['outSaveFolderPath'] = './notebook/T3T4/models/RNN/'
os.chdir('../../')#to the root path of autoBioSeqpy


#parameters
dataTypeList = paraDict['dataType']
dataEncodingType = paraDict['dataEncodingType']
spcLen = paraDict['spcLen']
firstKernelSize = paraDict['firstKernelSize']

dataTestFilePaths = paraDict['dataTestFilePaths']
dataTestLabel = paraDict['dataTestLabel']

dataSplitScale = paraDict['dataSplitScale']
outSaveFolderPath = paraDict['outSaveFolderPath']
showFig = paraDict['showFig']
```

```
saveFig = paraDict['saveFig']
savePrediction = paraDict['savePrediction']

loss = paraDict['loss']
optimizer = paraDict['optimizer']
if not optimizer.startswith('optimizers.'):
    optimizer = 'optimizers.' + optimizer
if not optimizer.endswith('()'):
    optimizer = optimizer + '()'
metrics = paraDict['metrics']

shuffleDataTrain = paraDict['shuffleDataTrain']
shuffleDataTest = paraDict['shuffleDataTest']
batch_size = paraDict['batch_size']
epochs = paraDict['epochs']

inputLength = paraDict['inputLength']
modelSaveName = paraDict['modelSaveName']
weightSaveName = paraDict['weightSaveName']
noGPU = paraDict['noGPU']
labelToMat = paraDict['labelToMat']


modelLoadFile = paraDict['modelLoadFile']
useKMerList = paraDict['useKMer']
if len(useKMerList ) == 0:
    useKMerList = [False] * len(modelLoadFile)
KMerNumList = paraDict['KMerNum']
if len(KMerNumList ) == 0:
    KMerNumList = [3] * len(modelLoadFile)

dataTestModelInd = paraDict['dataTestModelInd']
if len(modelLoadFile) == 1:
    dataTestModelInd = [0] * len(dataTestFilePaths)

modelPredictFile = outSaveFolderPath + os.path.sep + modelSaveName
weightLoadFile = outSaveFolderPath + os.path.sep + weightSaveName


model = moduleRead.
 ↪readModelFromJsonFileDirectly(modelPredictFile,weightLoadFile)
```

Then we begin to generate the dataset by encoding the sequences.

The code below is from predicting.py, to see how the code works, please see the other notebook 'tutorial in jupyter notebook.ipynb' instead.

[5]:
```
td = TextDecorate()

verbose = paraDict['verbose']

predictionSavePath = None
for i,k in enumerate(sys.argv):
    if k == '--predictionSavePath':
        predictionSavePath = sys.argv[i+1]
    elif k == '--verbose':
        verbose = sys.argv[i+1]

colorText = paraDict['colorText']
if colorText.lower() == 'auto':
    import platform
    if 'win' in platform.system().lower():
        td.disable()
elif not bool(eval(colorText)):
    td.disable()

if noGPU:
    if verbose:
        td.printC('As set by user, gpu will be disabled.','g')
    os.environ["CUDA_VISIBLE_DEVICES"] = '-1'
else:
    #check the version of tensorflow before configuration
    tfVersion = tf.__version__
    if int(tfVersion.split('.')[0]) >= 2:
        config = tf.compat.v1.ConfigProto()
        config.gpu_options.allow_growth = True

        sess =tf.compat.v1.Session(config=config)
    else:
        config = tf.ConfigProto(gpu_options=tf.GPUOptions(allow_growth=True))
        sess = tf.Session(config=config)


if not len(dataTypeList) == len(modelLoadFile):
    if verbose:
        td.printC('Please provide enough data type(s) as the number of␣
 ↪--modelLoadFile','r')
assert len(dataTypeList) == len(modelLoadFile)

featureGenerators = []
for i,subDataType in enumerate(dataTypeList):
    if subDataType.lower() == 'protein':
        if verbose:
            td.printC('Enconding protein data for model %d ...' %i,'b')
```

```
        featureGenerator = dataProcess.
↪ProteinFeatureGenerator(dataEncodingType[i], useKMer=useKMerList[i],␣
↪KMerNum=KMerNumList[i])
    elif subDataType.lower() == 'dna':
        if verbose:
            td.printC('Enconding DNA data for model %d ...' %i,'b')
        featureGenerator = dataProcess.DNAFeatureGenerator(dataEncodingType[i],␣
↪useKMer=useKMerList[i], KMerNum=KMerNumList[i])
    elif subDataType.lower() == 'rna':
        if verbose:
            td.printC('Enconding RNA data for model %d ...' %i,'b')
        featureGenerator = dataProcess.RNAFeatureGenerator(dataEncodingType[i],␣
↪useKMer=useKMerList[i], KMerNum=KMerNumList[i])
    elif subDataType.lower() == 'other':
        if verbose:
            td.printC('Reading CSV-like data for model %d ...' %i,'b')
        featureGenerator = dataProcess.OtherFeatureGenerator()
    else:
        td.printC('Unknow dataType %r, please use \'protein\', \'dna\' ,\'rna\'␣
↪or \'other\'' %subDataType, 'r')
    featureGenerators.append(featureGenerator)
    assert subDataType.lower() in ['protein','dna','rna','other']

if verbose:
    td.printC('Checking the number of test files, which should be larger than 1␣
↪(e.g. at least two labels)...','b')
assert len(dataTestFilePaths) > 0

if verbose:
    td.printC('Begin to generate test dataset...','b')

testDataLoadDict = {}
for modelIndex in range(len(modelLoadFile)):
    testDataLoadDict[modelIndex] = []
#    testDataLoaders = []
for i,dataPath in enumerate(dataTestFilePaths):
    modelIndex = dataTestModelInd[i]
    featureGenerator = featureGenerators[modelIndex]
    dataLoader = dataProcess.DataLoader(label = dataTestLabel[i],␣
↪featureGenerator=featureGenerator)
    dataLoader.readFile(dataPath, spcLen = spcLen[modelIndex])
    testDataLoadDict[modelIndex].append(dataLoader)

testDataMats = []
testLabelArrs = []
testNameLists = []
```

```
for modelIndex in range(len(modelLoadFile)):
    testDataLoaders = testDataLoadDict[modelIndex]
    testDataSetCreator = dataProcess.DataSetCreator(testDataLoaders)
    testDataMat, testLabelArr, nameList = testDataSetCreator.
 ↪getDataSet(toShuffle=False, withNameList=True)
    testDataMats.append(testDataMat)
    testLabelArrs.append(testLabelArr)
    testNameLists.append(nameList)
if verbose:
    td.printC('Test datasets generated.','g')
nameTemp = testNameLists[0]
testDataMats, testLabelArrs, sortedIndexes = dataProcess.
 ↪matAlignByName(testDataMats,nameTemp,testLabelArrs,testNameLists)
testNameLists = [nameTemp] * len(testNameLists)

tmpTempLabel = testLabelArrs[0]
for tmpLabel in testLabelArrs:
    assert np.sum(np.array(tmpTempLabel) - np.array(tmpLabel)) == 0

if labelToMat:
    if verbose:
        td.printC('Since labelToMat is set, the labels would be changed to␣
 ↪onehot-like matrix','g')
    testLabelArr,testLabelArrDict,testArrLabelDict = dataProcess.
 ↪labelToMat(testLabelArrs[0])
#    print(testLabelArr)
else:
    testLabelArr = testLabelArrs[0]
```

```
Enconding protein data for model 0 …
Checking the number of test files, which should be larger than 1 (e.g. at least
two labels)…
Begin to generate test dataset…
Test datasets generated.
```

[6]:
```
print('%d matrix generated:\n' %(len(testDataMats)))
for i in range(len(testDataMats)):
    print('Matrix and label %d:' %i)
    print(testDataMats[i])
    print(testLabelArrs[i])
    print('\n')
```

```
1 matrix generated:

Matrix and label 0:
[[12 18 17 … 20 20 20]
 [12  5  7 … 20 20 20]
```

```
 [12 10  3 … 20 20 20]
 …
 [12 13 11 … 20 20 20]
 [12  3  5 … 20 20 20]
 [12  1  1 … 20 20 20]]
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

1.3 model loading

Using the moduleRead module could load the model easily, then the components are ready.

[7]:
```
if verbose:
    td.printC('Checking module file for modeling','b')
if modelPredictFile is None:
    if verbose:
        td.printC('please provide a model file in a json file.','r')
if weightLoadFile is None:
    if verbose:
        td.printC('the weight file is necessary for predicting, otherwise the␣
 ↪model will be with initialized weight','r')
assert not modelPredictFile is None
assert not weightLoadFile is None

if verbose:
    td.printC('Loading module and weight file','b')
model = moduleRead.
 ↪readModelFromJsonFileDirectly(modelPredictFile,weightLoadFile)
if verbose:
    td.printC('Module loaded, generating the summary of the module','b')
    model.summary()
```

```
Checking module file for modeling
Loading module and weight file
Module loaded, generating the summary of the module
Model: "sequential_1"
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding_1 (Embedding)      (None, 1500, 256)         6656
_________________________________________________________________
bidirectional_1 (Bidirection (None, 128)               164352
_________________________________________________________________
dropout_1 (Dropout)          (None, 128)               0
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 129
=================================================================
Total params: 171,137
Trainable params: 171,137
Non-trainable params: 0
_________________________________________________________________
```

The model will be different as user set. Some times there will be a "sequential" layer, which is a wrapped model, in the summary, this time, we can umpack the "sequential" layer into 'un-wrapped' layers.

We provided few functions in 'layPlot.py', but here we can do it manually.

First of all, let us see what is in the sequential. A function is used to make this process more simple.

[8]:
```
def unBoundLayers(modelIn,layers = []):
    for layer in modelIn.layers:
        if not 'sequential' in layer.name.lower():
            layers.append(layer)
        else:
            unBoundLayers(layer,layers)
    return layers
```

[9]:
```
layers = unBoundLayers(model)
for i, layer in enumerate(layers):
    print('layer %d: %s' %(i,layer.name))
```

```
layer 0: embedding_1
layer 1: bidirectional_1
layer 2: dropout_1
layer 3: dense_1
```

Then, we need to generate a new model which change the output layer to a hidden layer. If the model contain multiple input layer, usually only the layer after concatenate is available.

[10]:
```
newModel = Model(inputs=model.input,outputs=model.layers[-2].output)
```

Now the model above is available for UMAP.

NOTE: Usually we have to spesify the input shape of the input layer, but autoBioSeqpy will check when training, thus here we don't have to do this step.

2. Using UMAP for layer visualization

UMAP need the predicted values from the model and the labels from the data, then the figure could be generated. Here the testDataMats is already changed to the training data.

Since the output layer is changed, the shape will be the same as the output of the such layer.

[11]:
```
predicted_Probability = newModel.predict(testDataMats[0])
```

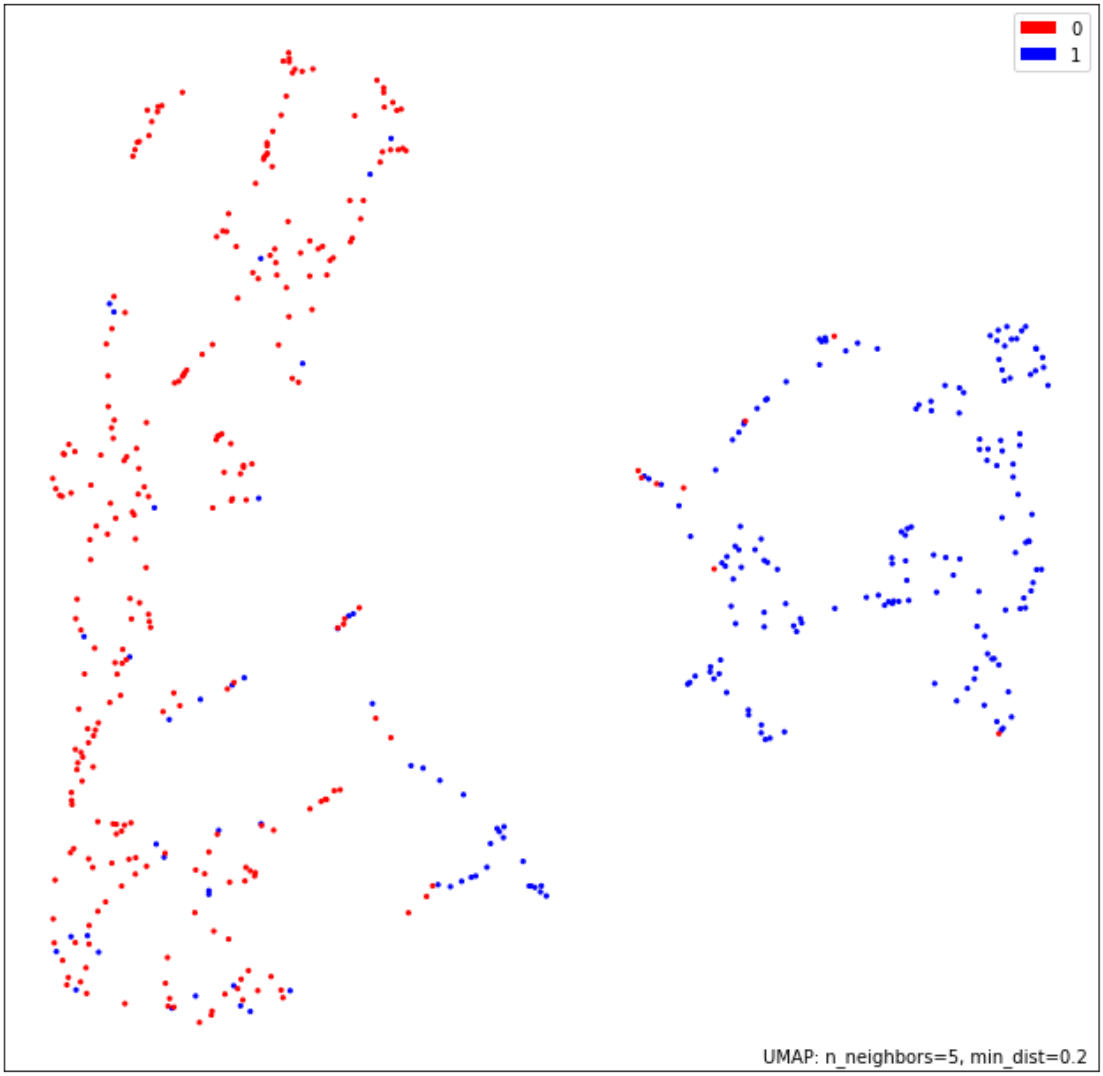
[12]:
```
predicted_Probability.shape
```

[12]:
```
(474, 128)
```

Using UMAP could get the representation that if the data are divided correctly, but still we might want to know the points which in the incorrect cluster, thus here we can export the axis to get the details.

[13]:
```
# the featureDict could contain more parameters, use '?umap.UMAP' for more␣
↪details.
featureDict={
        'n_neighbors' : 5,
        'min_dist' : 0.2,
        'n_components' : 2,
        'metric' : 'euclidean',
        }

mapper = umap.UMAP(**featureDict).fit(predicted_Probability)
plotObj = umap.plot.points(mapper, labels=testLabelArr, color_key=['r','b'])
plt.show()
```




One way is to use the umap.plot.show to see the polts interactively, a map of the label can make the visualization better.

[14]:
```
import pandas as pd
```

[15]:
```
hover_data = pd.DataFrame({'index':testDataLoaders[0].names+testDataLoaders[1].
 ↪names,
                           'label':testLabelArr})
```

[16]:
```
p = umap.plot.interactive(mapper, labels=testLabelArr, hover_data=hover_data,␣
 ↪point_size=2)
umap.plot.show(p)
```

Another way is to set some rule to find out the outliers manually. Using embedding_ in a UMAP

mapper can get the axis.

[17]:
```
mapper.embedding_
```

[17]:
```
array([[ 8.20463657e+00,  6.35753679e+00],
       [ 1.17314634e+01,  3.13039112e+00],
       [ 1.25600624e+01,  4.34175539e+00],
       [ 9.55835056e+00,  7.75112629e+00],
       [ 8.66233730e+00,  3.92251372e+00],
       [ 1.23687057e+01,  6.44397306e+00],
       [ 8.31627846e+00,  5.13510561e+00],
       [ 5.25744057e+00,  6.15125149e-02],
       [ 5.50547600e+00, -7.85565600e-02],
       [ 2.98458004e+00,  2.63535976e+00],
       [ 6.14167750e-01, -4.85858554e-03],
       [ 1.08655186e+01,  6.79454279e+00],
       [ 1.05176334e+01,  4.61093760e+00],
       [ 8.70639706e+00,  6.93512058e+00],
       [ 8.18388748e+00,  4.01000595e+00],
       [ 1.33686924e+00,  5.53207731e+00],
       [ 2.70693564e+00,  3.90322280e+00],
       [ 2.64226890e+00,  3.87274289e+00],
       [ 1.21604862e+01,  3.95611286e+00],
       [ 1.18616323e+01,  3.58718491e+00],
       [ 1.21431804e+00, -1.70918024e+00],
       [ 7.42812109e+00,  5.42783833e+00],
       [ 8.43705559e+00,  2.47500014e+00],
       [ 8.68523407e+00,  6.92163515e+00],
       [ 4.22385544e-01, -1.49104798e+00],
       [ 1.09041729e+01,  6.85186243e+00],
       [ 9.13260555e+00,  3.65144730e+00],
       [ 8.66281033e+00,  4.65657949e+00],
       [ 3.25517845e+00,  1.06094980e+01],
       [ 1.05204554e+01,  4.04851770e+00],
       [ 4.64493036e+00,  3.25635761e-01],
       [ 1.14887533e+01,  6.73453999e+00],
       [ 7.90026933e-02, -1.66222632e+00],
       [ 1.22485056e+01,  2.44650698e+00],
       [ 1.21801939e+01,  7.95407629e+00],
       [ 1.00178165e+01,  7.72302055e+00],
       [ 1.10867090e+01,  6.76896143e+00],
       [-5.33472896e-01,  3.29356337e+00],
       [ 8.61864281e+00,  4.95814800e+00],
       [ 4.27711010e+00,  1.28059000e-01],
       [ 1.22023726e+01,  2.80093312e+00],
       [ 1.10803738e+01,  6.90133953e+00],
       [ 1.04082184e+01,  4.02243423e+00],
```

```
[ 7.55430460e+00,  2.90966368e+00],
[ 3.54436922e+00,  1.75983965e+00],
[-1.49249598e-01,  6.52316451e-01],
[ 1.25038824e+01,  4.92193937e+00],
[ 1.21496496e+01,  7.68087101e+00],
[ 1.20638962e+01,  3.18192124e+00],
[ 8.21374893e+00,  4.39224243e+00],
[ 8.68006516e+00,  2.12849784e+00],
[ 8.98101616e+00,  7.17768335e+00],
[ 1.20539684e+01,  7.90263224e+00],
[ 8.56067848e+00,  6.80029058e+00],
[ 1.05123491e+01,  4.69114494e+00],
[-1.38452089e+00, -6.52844906e-01],
[ 9.49606514e+00,  7.74348545e+00],
[ 1.23949356e+01,  7.89649200e+00],
[ 8.29509163e+00,  6.46254206e+00],
[ 7.59481764e+00,  4.99406862e+00],
[ 1.16083097e+01,  2.85031700e+00],
[ 1.07129354e+01,  4.65030813e+00],
[ 8.24213982e+00,  4.85581160e+00],
[ 8.37585258e+00,  6.61113548e+00],
[ 9.73181009e-01, -1.34551096e+00],
[ 7.66309690e+00,  3.02262402e+00],
[ 8.10073090e+00,  4.57424974e+00],
[ 4.89204025e+00,  8.97707105e-01],
[ 1.24493732e+01,  3.98603415e+00],
[ 1.20372458e+01,  7.75608635e+00],
[ 1.25956774e+01,  7.33160019e+00],
[ 1.24966183e+01,  4.93335962e+00],
[ 1.20353146e+01,  2.38183713e+00],
[ 1.18998861e+01,  3.33773971e+00],
[ 9.18616581e+00,  3.83439302e+00],
[ 1.22702675e+01,  6.01384211e+00],
[ 9.46493912e+00,  7.41779661e+00],
[ 1.25240993e+01,  7.27956295e+00],
[ 1.20346107e+01,  5.99349308e+00],
[ 1.25437479e+01,  5.30531263e+00],
[ 1.07385874e+01,  5.10483503e+00],
[ 4.91654426e-01,  2.69466662e+00],
[ 3.96164083e+00,  1.55058348e+00],
[ 9.08951473e+00,  3.73203111e+00],
[ 1.21256657e+01,  2.28950429e+00],
[ 1.19970903e+01,  3.26886296e+00],
[ 1.23436499e+01,  5.58687019e+00],
[ 1.06508532e+01,  5.05881548e+00],
[-1.19543290e+00,  3.58101392e+00],
[ 4.41721106e+00,  1.84298128e-01],
```

```
[ 6.92455196e+00,  5.84374857e+00],
[ 1.17868853e+01,  6.36482763e+00],
[ 1.97242391e+00,  7.43382788e+00],
[ 1.11333265e+01,  2.91839647e+00],
[ 1.26984987e+01,  7.51867199e+00],
[ 2.48121572e+00,  3.69391751e+00],
[ 7.93325424e+00,  2.98552561e+00],
[ 7.95804310e+00,  5.93188858e+00],
[ 1.17102737e+01,  2.98541355e+00],
[ 1.03036251e+01,  7.64254379e+00],
[ 4.11497211e+00,  5.22268862e-02],
[ 3.95521410e-02,  2.41062450e+00],
[ 8.62177467e+00,  2.33491564e+00],
[ 1.26056700e+01,  7.63929510e+00],
[ 1.01449032e+01,  4.13465118e+00],
[ 1.26825085e+01,  4.52909994e+00],
[ 1.17483969e+01,  3.72446990e+00],
[ 9.46460056e+00,  7.77810621e+00],
[ 1.23741417e+01,  3.97536516e+00],
[ 1.19065609e+01,  6.22709036e+00],
[ 1.21136589e+01,  6.20117092e+00],
[ 8.33174706e+00,  4.55887651e+00],
[ 1.06135588e+01,  4.07671595e+00],
[ 8.66403866e+00,  3.71811080e+00],
[ 4.82441139e+00,  8.27572465e-01],
[ 7.16970253e+00,  5.72736835e+00],
[ 1.24544420e+01,  4.90817165e+00],
[-3.44609916e-02,  4.68343824e-01],
[ 8.01003838e+00,  3.05657530e+00],
[-1.14551437e+00, -6.38864994e-01],
[ 1.24259615e+01,  4.12492132e+00],
[-8.25917244e-01,  8.27710724e+00],
[ 1.14983711e+01,  4.67638969e+00],
[ 1.07945499e+01,  5.12886190e+00],
[ 8.29517269e+00,  4.80585718e+00],
[ 5.42018175e+00, -2.20446810e-02],
[ 9.68690205e+00,  3.97844315e+00],
[ 1.12947893e+01,  4.69648981e+00],
[ 1.25313654e+01,  4.22588539e+00],
[ 9.84564590e+00,  7.61132479e+00],
[ 4.79178762e+00,  8.72563660e-01],
[ 6.99060917e+00,  5.80744314e+00],
[-7.62008786e-01,  8.16150856e+00],
[ 1.20664310e+01,  7.49169779e+00],
[ 1.07577038e+01,  4.09112453e+00],
[ 1.07297421e+01,  4.37445116e+00],
[ 7.58507109e+00,  2.93328881e+00],
```

```
[ 1.14928055e+01,  7.09021854e+00],
[ 1.26114378e+01,  4.52617645e+00],
[ 2.95111918e+00,  1.01041470e+01],
[ 1.12700295e+00,  2.99977088e+00],
[ 1.03179626e+01,  4.16411209e+00],
[ 8.61901855e+00,  2.22217774e+00],
[ 3.93559813e+00,  7.97149092e-02],
[ 1.21098166e+01,  5.25861883e+00],
[ 5.34057903e+00,  4.23300602e-02],
[ 1.79164588e+00, -1.41578186e+00],
[ 8.11763000e+00,  2.79206967e+00],
[ 1.04659548e+01,  4.07308340e+00],
[ 9.20689392e+00,  3.77355289e+00],
[ 1.27123919e+01,  7.37971640e+00],
[ 1.36709833e+00,  8.91586590e+00],
[ 7.88474655e+00,  3.15423942e+00],
[-1.31157017e+00, -1.40370297e+00],
[ 1.23667021e+01,  6.27935600e+00],
[ 1.12837429e+01,  7.12456083e+00],
[ 8.05123425e+00,  4.61986589e+00],
[ 8.36805439e+00,  6.58055210e+00],
[ 1.17737885e+01,  4.21677160e+00],
[ 8.74631786e+00,  2.14723492e+00],
[ 3.71939015e+00,  1.72479558e+00],
[ 8.24929047e+00,  3.76358581e+00],
[ 1.26029053e+01,  7.64941025e+00],
[ 7.56433070e-01,  8.46429169e-01],
[-1.59647858e+00, -8.64143133e-01],
[ 1.05388794e+01,  4.08280087e+00],
[ 9.52959299e-01,  2.89743400e+00],
[ 1.21441975e+01,  7.19167852e+00],
[ 8.43454742e+00,  2.54422760e+00],
[ 1.37116385e+00,  9.40940142e-01],
[ 1.15544987e+01,  7.02402687e+00],
[ 8.85418892e+00,  4.52956629e+00],
[ 1.17845869e+01,  6.22274351e+00],
[ 1.18527832e+01,  2.61180782e+00],
[ 1.07095394e+01,  5.01138210e+00],
[ 5.43331766e+00,  6.43402636e-02],
[ 1.23047380e+01,  4.76500416e+00],
[ 1.23227291e+01,  7.78194094e+00],
[ 1.19397373e+01,  7.82746601e+00],
[ 5.16836452e+00,  4.08665448e-01],
[ 1.07323718e+00, -1.62918150e+00],
[ 8.03003025e+00,  3.25042558e+00],
[ 1.27761021e+01,  7.12521076e+00],
[ 1.11611967e+01,  4.20752621e+00],
```

```
[ 1.24534874e+01,  7.95504713e+00],
[ 5.28514481e+00,  6.03572465e-02],
[ 1.22272625e+01,  7.22832203e+00],
[-1.75509468e-01,  5.39887619e+00],
[ 1.20993195e+01,  2.26109719e+00],
[ 1.18926983e+01,  6.06650496e+00],
[ 8.97474003e+00,  4.04408264e+00],
[ 8.95596695e+00,  2.23644638e+00],
[ 4.69644499e+00,  6.59649968e-01],
[ 1.20910730e+01,  6.34920454e+00],
[ 8.12331200e+00,  4.70909214e+00],
[ 1.14531813e+01,  2.67308092e+00],
[ 9.54626274e+00,  7.79681540e+00],
[ 7.87844801e+00,  3.08308077e+00],
[ 6.13727272e-01, -5.61146662e-02],
[ 1.22471352e+01,  7.77738476e+00],
[ 4.30459642e+00,  1.35036838e+00],
[-9.83527184e-01, -8.72652233e-01],
[ 8.71072197e+00,  4.62818480e+00],
[ 1.21114416e+01,  7.34386492e+00],
[ 4.48249340e+00,  2.02258855e-01],
[ 1.19653406e+01,  3.26538563e+00],
[ 1.22713346e+01,  5.83278465e+00],
[ 1.11194344e+01,  4.73317003e+00],
[ 8.53048325e+00,  4.80795908e+00],
[ 4.88300037e+00,  7.44991541e-01],
[ 1.14045153e+01,  4.44751930e+00],
[ 1.76995993e+00,  1.18192492e+01],
[ 2.07554817e+00,  8.66804028e+00],
[ 1.53983736e+00,  1.07100430e+01],
[ 2.92773068e-01,  7.34149075e+00],
[-1.32019353e+00,  1.98943794e+00],
[-1.12852001e+00,  4.45119590e-01],
[-4.69102561e-01,  5.29530287e+00],
[ 1.21562541e+00,  2.74405390e-01],
[-2.77979374e-01,  1.10087299e+01],
[-2.61442542e-01,  5.55697441e+00],
[ 9.40226555e-01,  5.49884987e+00],
[ 1.32850850e+00,  8.63086605e+00],
[-7.97732115e-01,  6.52682257e+00],
[ 6.49799526e-01, -1.75938439e+00],
[-1.32196486e+00, -7.37426221e-01],
[ 1.82323527e+00,  7.22430229e+00],
[-1.16661680e+00, -1.09304905e+00],
[-9.95996118e-01,  9.69212830e-01],
[ 2.43125296e+00,  1.40547132e+00],
[-2.90632576e-01,  6.60112810e+00],
```

```
[ 2.32084751e+00,  1.28009701e+00],
[ 1.03349853e+00,  8.35481548e+00],
[ 8.00941110e-01,  6.44004917e+00],
[ 3.14812660e+00,  1.12581110e+01],
[ 6.84606016e-01, -5.75047016e-01],
[ 1.08207643e+00, -1.25368190e+00],
[ 2.72818518e+00,  1.09314337e+01],
[ 1.76217365e+00,  9.43794632e+00],
[ 1.68780959e+00, -1.51476693e+00],
[ 2.31157914e-01,  7.25167036e+00],
[-5.01454510e-02,  2.52128983e+00],
[ 2.41710401e+00,  8.92542839e+00],
[ 2.12126040e+00,  1.15904503e+01],
[ 3.76906157e+00, -8.65213051e-02],
[ 8.35004508e-01,  5.89995050e+00],
[-2.31419578e-01,  3.71105766e+00],
[ 1.50210094e+00,  8.95962048e+00],
[ 2.66547894e+00,  9.15113258e+00],
[ 9.33043718e-01,  6.30311441e+00],
[ 2.07775545e+00,  9.16410542e+00],
[-6.02459013e-01, -1.59797144e+00],
[-7.56762266e-01,  6.63530445e+00],
[ 6.65883422e-01,  7.70174122e+00],
[ 1.13650143e+00,  1.32491067e-01],
[ 3.25191593e+00,  2.15447521e+00],
[ 2.30157763e-01,  1.12606316e+01],
[ 1.10610116e+00, -1.54653192e+00],
[-1.12106133e+00, -4.95907396e-01],
[-1.64842176e+00,  5.81113577e+00],
[ 1.93829298e-01,  2.60615873e+00],
[ 1.62503517e+00,  7.69677734e+00],
[ 2.26193666e+00,  9.08731937e+00],
[-6.08282626e-01,  9.19765353e-01],
[-1.23845208e+00,  1.94518614e+00],
[-1.01717365e+00,  5.13907671e+00],
[-5.99969983e-01,  8.15276527e+00],
[ 1.20648937e+01,  2.21130967e+00],
[ 3.85817289e+00,  6.36897758e-02],
[ 3.50686049e+00, -3.16308528e-01],
[-6.50963962e-01,  8.36290061e-01],
[ 3.15718484e+00,  1.10515938e+01],
[ 1.40683401e+00,  1.03159904e+01],
[ 1.25109434e+00,  8.71639729e+00],
[ 2.85405755e+00,  9.73410702e+00],
[ 2.48268962e+00,  3.70492673e+00],
[-1.36857176e+00,  1.21109390e+00],
[-5.27570605e-01,  4.11713648e+00],
```

```
[-1.62700081e+00, -7.41943479e-01],
[-3.91024411e-01,  4.05801010e+00],
[-1.50630939e+00, -9.88291800e-01],
[-4.94407088e-01,  5.33744669e+00],
[-1.21601932e-01,  1.10593023e+01],
[ 1.18833447e+00, -1.13245118e+00],
[ 3.40112877e+00,  1.10230598e+01],
[ 3.42537069e+00,  1.04696026e+01],
[-7.92593002e-01,  3.84044975e-01],
[-9.87235248e-01,  2.36417913e+00],
[ 8.99392366e-01,  9.54805756e+00],
[-9.52484012e-01,  6.14171028e+00],
[-7.60826468e-01,  8.37904930e+00],
[-1.04065168e+00,  3.41756988e+00],
[ 3.09632945e+00,  1.02754526e+01],
[-1.54782355e+00,  5.57559919e+00],
[-8.47892761e-01,  7.25803518e+00],
[ 1.05040558e-01,  2.78558230e+00],
[-6.70406297e-02,  1.10715094e+01],
[ 1.58464718e+00,  8.85636711e+00],
[-1.31386533e-01,  1.09871168e+01],
[ 7.93766212e+00,  4.53535414e+00],
[-4.49013859e-01,  4.95888758e+00],
[ 6.59597814e-01, -1.70460546e+00],
[ 2.69291377e+00,  9.20173264e+00],
[-7.74236917e-01,  9.38836336e-01],
[-4.22720939e-01,  1.05551672e+01],
[ 1.77859128e+00,  1.16898928e+01],
[ 1.77965567e-01,  7.18276405e+00],
[ 2.58743882e-01,  7.29965353e+00],
[-1.05747354e+00,  2.18226743e+00],
[-1.11011016e+00,  4.94819164e+00],
[-7.72500455e-01,  6.37928963e+00],
[ 7.38871276e-01,  6.40690231e+00],
[ 6.83746815e+00,  5.92271185e+00],
[-7.15243459e-01,  3.05039263e+00],
[ 2.32366061e+00,  8.68257236e+00],
[-1.31496358e+00,  3.83400559e+00],
[-1.37510872e+00,  1.26897478e+00],
[ 1.77936780e+00,  1.17387047e+01],
[ 1.73744452e+00,  8.50464916e+00],
[-1.06895697e+00,  3.23275119e-01],
[-8.55455577e-01,  5.02662468e+00],
[ 1.46497544e-02, -9.48035181e-01],
[-1.15786791e+00, -1.45452690e+00],
[ 7.10478020e+00,  5.73633957e+00],
[ 1.69218266e+00,  1.17015009e+01],
```

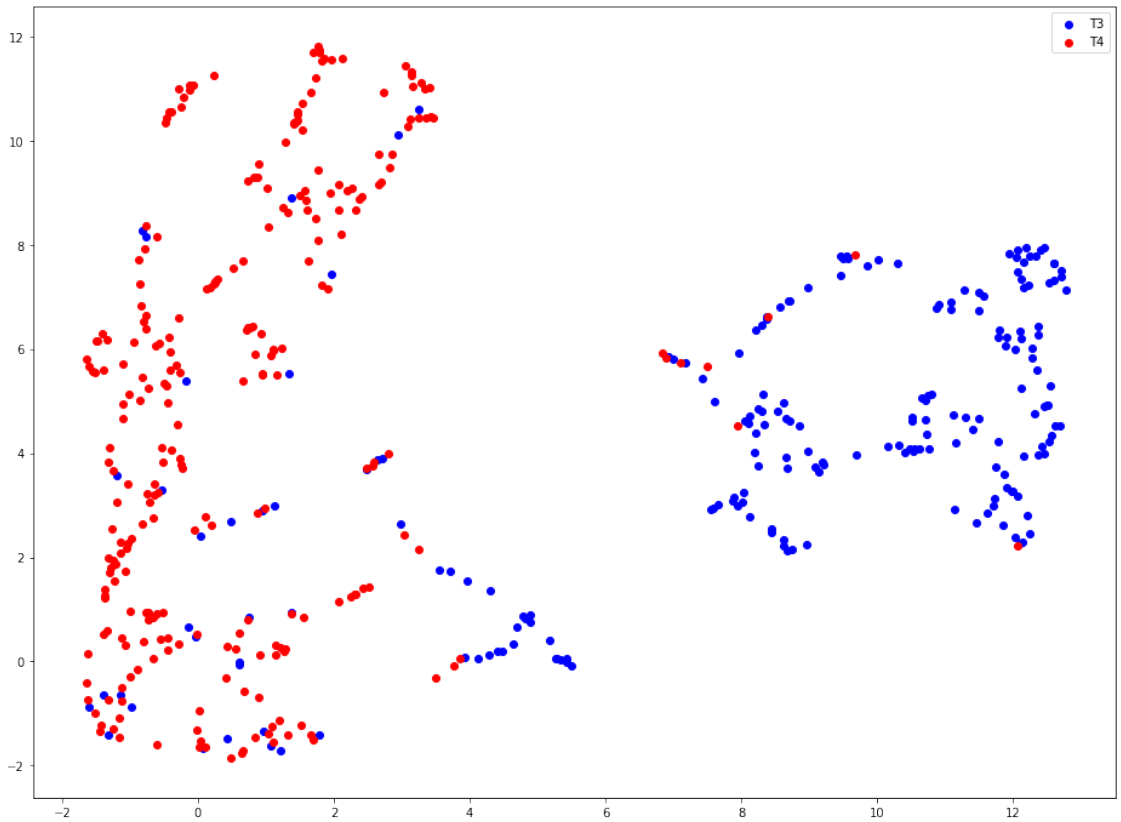
```
[-2.06314381e-02,  5.23323774e-01],
[-4.10632372e-01,  5.59084368e+00],
[ 2.56784201e+00,  3.75683951e+00],
[-1.42219281e+00, -1.22825384e+00],
[-7.45970130e-01,  3.21262765e+00],
[-2.95981526e-01,  4.55454826e+00],
[ 1.86112559e+00,  1.15880556e+01],
[ 1.73562515e+00,  1.12063990e+01],
[-1.12299073e+00, -7.57933497e-01],
[ 1.60001051e+00,  8.68137932e+00],
[ 1.28836000e+00,  2.49090299e-01],
[ 2.65766048e+00,  9.73347282e+00],
[ 1.51527178e+00, -1.21615601e+00],
[ 3.33780169e+00,  1.10054121e+01],
[ 3.13098335e+00,  1.04204483e+01],
[-5.44473410e-01,  4.39807802e-01],
[-2.60673612e-01,  3.89381528e+00],
[ 1.27671254e+00,  2.02234507e-01],
[-5.08106589e-01,  3.82997751e+00],
[ 1.11498594e+00,  5.99724150e+00],
[ 1.94192827e+00,  8.99534035e+00],
[ 7.49464226e+00,  5.67931128e+00],
[ 1.66171527e+00,  1.09386921e+01],
[-8.82452548e-01, -1.63479075e-01],
[ 1.45668113e+00,  1.05477819e+01],
[ 6.88459682e+00,  5.82202053e+00],
[-1.03112793e+00,  2.26324534e+00],
[ 2.51741481e+00,  1.41920698e+00],
[ 1.17782183e-01,  7.15962124e+00],
[ 4.23060060e-01,  2.88385093e-01],
[-3.22267205e-01,  5.69262838e+00],
[-9.95104969e-01, -2.95476019e-01],
[ 1.82668257e+00,  1.15355196e+01],
[ 3.35939145e+00,  1.04490585e+01],
[ 2.11031246e+00,  8.19630718e+00],
[ 1.44879305e+00,  1.03794403e+01],
[ 7.36469924e-01,  7.91914165e-01],
[-2.14248672e-01,  1.08467569e+01],
[-7.26845086e-01,  9.32356596e-01],
[-7.26370990e-01,  7.96159565e-01],
[ 9.00099039e-01, -6.87734783e-01],
[-6.14547551e-01,  6.06370258e+00],
[-7.91477561e-01,  7.92591763e+00],
[-6.42173171e-01,  3.20388651e+00],
[ 1.53200579e+00,  1.02133417e+01],
[ 2.25255545e-02, -1.63482928e+00],
[-1.41484416e+00,  6.29211664e+00],
```

```
[ 1.15047348e+00,  3.23516160e-01],
[-6.35087788e-01,  3.39943171e+00],
[ 3.14723349e+00,  1.13231077e+01],
[ 8.39036369e+00,  6.62266207e+00],
[-1.60421634e+00,  5.66685867e+00],
[-1.48061979e+00,  6.15031862e+00],
[-1.24550784e+00, -1.29195368e+00],
[ 1.41711450e+00,  1.03499975e+01],
[-4.56755638e-01,  1.04507275e+01],
[-1.61547661e+00,  1.45970955e-01],
[ 2.46991590e-01,  7.27086639e+00],
[ 1.15526712e+00,  5.50863314e+00],
[-5.81590533e-01,  3.25640178e+00],
[-1.24009860e+00,  3.67330551e+00],
[-2.48182505e-01,  3.79240227e+00],
[ 2.79492188e+00,  3.98773479e+00],
[ 1.96643436e+00,  1.15562391e+01],
[ 8.46831262e-01, -1.46399522e+00],
[-1.22266507e+00,  1.54231572e+00],
[ 1.01118386e+00,  9.08438587e+00],
[-4.86770511e-01,  1.03583574e+01],
[-6.50342047e-01,  5.97894751e-02],
[-4.34702456e-01,  6.23545599e+00],
[-2.53993243e-01,  1.06503572e+01],
[-1.38396299e+00,  5.60717821e+00],
[-8.22002828e-01,  2.65028238e+00],
[-5.76851606e-01,  6.11887550e+00],
[ 1.55281687e+00,  8.48461509e-01],
[-1.34463155e+00,  5.89525998e-01],
[ 1.32313943e+00, -1.41529644e+00],
[ 4.48155627e-02, -1.52517164e+00],
[-1.06319499e+00,  1.74258053e+00],
[ 4.11125958e-01, -3.13782275e-01],
[ 3.28013229e+00,  1.11177635e+01],
[ 2.81371450e+00,  9.47480297e+00],
[ 3.24681139e+00,  1.04488621e+01],
[ 9.67896652e+00,  7.81916714e+00],
[-8.11443567e-01,  5.46155119e+00],
[ 9.14814651e-01,  1.15850255e-01],
[ 3.03338885e+00,  2.42798376e+00],
[-4.45722014e-01,  4.60529864e-01],
[ 8.69760036e-01,  9.29593182e+00],
[ 7.21077740e-01,  6.35760880e+00],
[-1.10062277e+00,  4.66633511e+00],
[ 8.83060992e-01,  2.84244227e+00],
[ 2.24110055e+00,  1.25413048e+00],
[ 6.69914842e-01,  5.39680910e+00],
```

```
[ 1.07128966e+00,  5.87856054e+00],
[ 1.77508450e+00,  8.09534836e+00],
[ 9.78548646e-01,  2.93356681e+00],
[-5.16113758e-01,  9.51493800e-01],
[ 1.23890173e+00,  6.01696253e+00],
[-1.14337146e+00,  2.28066421e+00],
[-3.94741386e-01,  1.05666380e+01],
[-7.36695349e-01,  5.25113106e+00],
[-4.50536937e-01,  2.29153395e-01],
[ 9.55407798e-01,  5.52694035e+00],
[-1.18798435e+00,  3.05349565e+00],
[ 1.46094954e+00,  1.04024363e+01],
[-2.89265513e-01,  3.40373755e-01],
[ 1.11560118e+00,  5.96873188e+00],
[ 1.66040802e+00, -1.40724111e+00],
[-1.09584177e+00,  5.71810627e+00],
[ 1.91283703e+00,  7.16906595e+00],
[-6.66493297e-01,  2.75049996e+00],
[ 1.56994200e+00,  9.05021381e+00],
[-1.27065694e+00,  2.55893636e+00],
[-1.37841570e+00,  1.38116038e+00],
[ 1.37780035e+00,  9.16402519e-01],
[ 1.45774627e+00,  1.05069914e+01],
[ 2.58260822e+00,  3.83284187e+00],
[-3.99954766e-01,  5.95239401e+00],
[ 7.61889756e-01,  6.41734886e+00],
[ 1.08397983e-01, -1.64833939e+00],
[-1.13124371e+00,  2.08356571e+00],
[ 2.19791222e+00,  9.05104828e+00],
[ 8.15508723e-01,  9.30330276e+00],
[ 3.05057001e+00,  1.14319410e+01],
[ 7.27267146e-01,  9.22307205e+00],
[-1.29839635e+00,  1.70344698e+00],
[ 3.47168064e+00,  1.04371500e+01],
[-1.21362269e+00,  1.87936473e+00],
[ 5.53068817e-01,  2.32801914e-01],
[ 2.30060744e+00,  1.28426826e+00],
[-1.44442916e+00, -1.33356750e+00],
[ 1.03151262e+00, -1.38738930e+00],
[ 1.29216027e+00,  9.97471046e+00],
[-1.49546432e+00,  6.16215467e+00],
[-1.29587114e+00,  4.10877943e+00],
[-1.51118450e-02, -1.31117356e+00],
[-1.28637779e+00,  1.79790151e+00],
[-1.51470518e+00,  5.55882502e+00],
[-8.72705519e-01,  7.70903635e+00],
[-8.44110131e-01,  6.82763195e+00],
```

```
       [-1.64274979e+00, -4.02292430e-01],
       [ 2.36991358e+00,  8.88945770e+00],
       [ 4.77658361e-01, -1.86412776e+00],
       [-1.32835126e+00,  6.19141626e+00],
       [ 2.07977176e+00,  1.15010595e+00],
       [ 5.17422616e-01,  7.56303883e+00],
       [ 6.14153504e-01,  5.41899145e-01],
       [-1.39162648e+00,  5.30523956e-01]], dtype=float32)
```

[18]:
```
plt.figure(figsize=(16,12))
t3Num = testLabelArr.sum() #1:T3,0:T4
t3s = plt.scatter(mapper.embedding_[:t3Num, 0], mapper.embedding_[:t3Num, 1],␣
 ↪color='b')
t4s = plt.scatter(mapper.embedding_[t3Num:, 0], mapper.embedding_[t3Num:, 1],␣
 ↪color='r')
plt.legend((t3s, t4s),
           ('T3', 'T4'),
           #           loc='lower left',
           )
plt.show()
```



With the polt above, we can find that this time the two cluster could be roughly divided by y=6

[19]:
```
nameList = np.array(testDataLoaders[0].names+testDataLoaders[1].names)
```

[20]:
```
errList = []
tmpIndex = (mapper.embedding_[:,0] > 6).astype(int) * (np.array(testLabelArr)␣
↪== 0).astype(int)
errList += list(nameList[np.arange(len(tmpIndex))[tmpIndex.astype(np.bool)]])
tmpIndex = (mapper.embedding_[:,0] < 6).astype(int) * (np.array(testLabelArr)␣
↪== 1).astype(int)
errList += list(nameList[np.arange(len(tmpIndex))[tmpIndex.astype(np.bool)]])
```

[21]:
```
print(errList)
```

```
['trainT4.txt_id113', 'trainT4.txt_id177', 'trainT4.txt_id203',
'trainT4.txt_id225', 'trainT4.txt_id271', 'trainT4.txt_id279',
'trainT4.txt_id329', 'trainT4.txt_id395', 'trainT3.txt_id15',
'trainT3.txt_id17', 'trainT3.txt_id19', 'trainT3.txt_id21', 'trainT3.txt_id31',
'trainT3.txt_id33', 'trainT3.txt_id35', 'trainT3.txt_id41', 'trainT3.txt_id49',
'trainT3.txt_id57', 'trainT3.txt_id61', 'trainT3.txt_id65', 'trainT3.txt_id75',
'trainT3.txt_id79', 'trainT3.txt_id89', 'trainT3.txt_id91', 'trainT3.txt_id111',
'trainT3.txt_id129', 'trainT3.txt_id135', 'trainT3.txt_id163',
'trainT3.txt_id165', 'trainT3.txt_id177', 'trainT3.txt_id179',
'trainT3.txt_id185', 'trainT3.txt_id191', 'trainT3.txt_id201',
'trainT3.txt_id203', 'trainT3.txt_id229', 'trainT3.txt_id235',
'trainT3.txt_id239', 'trainT3.txt_id243', 'trainT3.txt_id251',
'trainT3.txt_id261', 'trainT3.txt_id265', 'trainT3.txt_id279',
'trainT3.txt_id281', 'trainT3.txt_id287', 'trainT3.txt_id291',
'trainT3.txt_id293', 'trainT3.txt_id303', 'trainT3.txt_id307',
'trainT3.txt_id321', 'trainT3.txt_id327', 'trainT3.txt_id329',
'trainT3.txt_id333', 'trainT3.txt_id339', 'trainT3.txt_id351',
'trainT3.txt_id359', 'trainT3.txt_id361', 'trainT3.txt_id371',
'trainT3.txt_id375', 'trainT3.txt_id385', 'trainT3.txt_id397',
'trainT3.txt_id401', 'trainT3.txt_id403', 'trainT3.txt_id409',
'trainT3.txt_id419']
```

That's all the informations for plotting the layer by using their outputs and UMAP. If you still have more issues, please fell free to connect us at ljs@swmu.edu.cn, thanks.